



OpenMP - Sincronização e Sections

Professora: Carla Santana
carla.santana@imd.ufrn.br

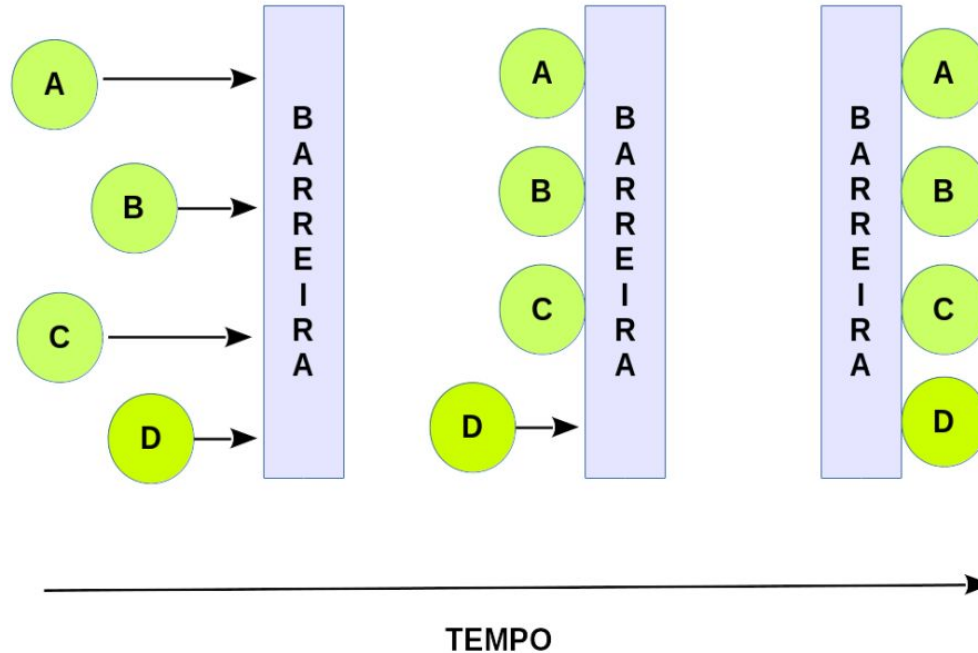


Sincronização

- Evitar condição de corrida
- Assegurar ordem correta de funções
- Diretivas
 - Critical
 - Atomic
 - Barrier
 - Master
 - Single
 - Funções de Lock

Diretiva Barrier

Garantir que certas **fases da computação terminaram**, antes de avançar na execução do programa.



Diretiva Barrier



```
# pragma omp barrier
```



```
# pragma omp parallel private (i , tid , vizinho )
{
    tid = omp_get_thread_num ( ) ;
    vizinho = tid - 1;
    if ( tid == 0)
        vizinho = omp_get_num_threads ( ) - 1;
    ...
    a[ tid ] = a [ tid ]*3.5;
    # pragma omp barrier
    b[ tid ] = a [ vizinho ] + c
    ...
}
```



Diretiva single

- Executado por uma única thread
- A primeira thread que alcançar a diretiva single executará o bloco.
- As outras threads do time devem esperar o final da execução do bloco
 - **Existe uma barreira implícita**



```
#pragma omp single [cláusulas]
{
    bloco estruturado
}
```

Diretiva single

```
#pragma omp single [cláusulas]
{
    bloco estruturado
}
```

```
#include <stdio.h>
#include <omp.h>
void trab1 ( int tid ) {
    printf ( " Executando trab1 na thread %d \n", tid ) ;
}
void trab2 ( int tid ) {
    printf ( " Executando trab2 na thread %d \n", tid ) ;
}
int main ( int argc , char * argv [] ) {
    #pragma omp parallel num_threads (4)
    {
        int tid = omp_get_thread_num () ;
        /* Trecho executado por uma única thread ,
        ,→ aleatoriamente */
        #pragma omp single
        trab1 ( tid ) ;
        /* Trecho executado por todas as threads , sempre
        ,→ depois de trab1 */
        trab2 ( tid ) ;
    }
    return (0) ;
}
```

```
Executando trab1 na thread 2
Executando trab2 na thread 1
Executando trab2 na thread 3
Executando trab2 na thread 0
Executando trab2 na thread 2
```





Diretiva Master

- Executado **apenas pela thread master (thread 0)**.
- As demais threads pulam o bloco estruturado e continuam a sua execução.
- Nesse aspecto, seu comportamento é diferente da diretiva single, **já que não existe uma barreira implícita ao final da região paralela**.
- Na maior parte das vezes é utilizada para operações de E/S.



```
#pragma omp master
{
    bloco estruturado
}
```

Diretiva Master



```
#pragma omp master
{
    bloco estruturado
}
```



```
# include < stdio .h >
# include < omp .h >
int main ( int argc , char * argv []) {
    int a [5] , i;
    # pragma omp parallel
    {
        // Realiza alguma computação
        # pragma omp for
        for (i = 0; i < 5; i ++ )
            a[i ] = i * i ;
        // Imprime resultados intermediários
        # pragma omp master
        for (i = 0; i < 5; i ++ )
            printf ("a[%d] = %d\n", i , a[i ]) ;
        // Espera
        # pragma omp barrier
        // Continua a computação
        # pragma omp for
        for (i = 0; i < 5; i ++ )
            a[i ] += i;
    }
}
```



Cláusula nowait

- A cláusula nowait pode suprimir as **barreiras implícitas no final das diretivas for, sections e single.**
- Aumentar o paralelismo



```
#pragma omp for nowait  
laço for
```



Cláusula nowait



```
#pragma omp for nowait  
laço for
```



```
# pragma omp parallel  
{  
    # pragma omp for nowait  
    for (j = 0; j < n; j ++ ) {  
        a[j ] = c * b [j ];  
    }  
    # pragma omp for nowait  
    for (i = 0; i < m; i ++ ) {  
        x[i ] = sqrt (y[i ]) * 2.0;  
    }  
}
```



Cláusula nowait

```
# pragma omp for nowait
for (j = 0; j < k; j++) {
    a[j] = b[j] + c[j];
}

# pragma omp for nowait
for (j = 0; j < m; j++) {
    d[j] = e[j] * f;
}

# pragma omp for nowait
for (j = 0; j < n; j++) {
    z[j] = (a[j] + a[j + 1]) * 0.5;
}
```



Cláusula nowait

Antes de chegar no
último loop
precisaremos de uma
barreira

```
# pragma omp for nowait
for (j = 0; j < k; j ++){
    a[j] = b[j] + c[j];
}

# pragma omp for nowait
for (j = 0; j < m; j ++){
    d[j] = e[j] * f;
}

# pragma omp for nowait
for (j = 0; j < n; j ++){
    z[j] = (a[j] + a[j + 1]) * 0.5;
}
```



Diretiva	Quem executa	Exclusão mútua	Barreira Implícita
<code>critical</code>	todas	sim	não
<code>atomic</code>	todas	sim	não
<code>single</code>	uma qualquer	não	sim
<code>master</code>	thread 0	não	não
<code>single nowait</code>	uma qualquer	não	não



Particionamento

- Um dos primeiros passos para a criação de um programa paralelo é quebrar o problema em partes de trabalho para serem distribuídas aos diferentes fluxos de execução.
- **Decomposição ou particionamento de dados e tarefas.**
- Extrair o máximo de paralelismo sem gerar uma sobrecarga significativa de gerenciamento das tarefas em relação ao trabalho útil a ser feito.
- **As tarefas de comunicação e sincronização não podem ser mais complexas do que as tarefas de computação.**
 - Se isso ocorrer, a execução em paralelo certamente será mais ineficiente do que a sequencial.



Tipo de Particionamento

- Depende do modelo de programação adotado e da eficiência das primitivas de comunicação e sincronização desse modelo na plataforma alvo
- Localidade dos dados
- Impacta na serialização no acesso a recursos compartilhados
- Impacta no custo de comunicação e da sincronização visto pelos processadores



Particionamento de dados e tarefas

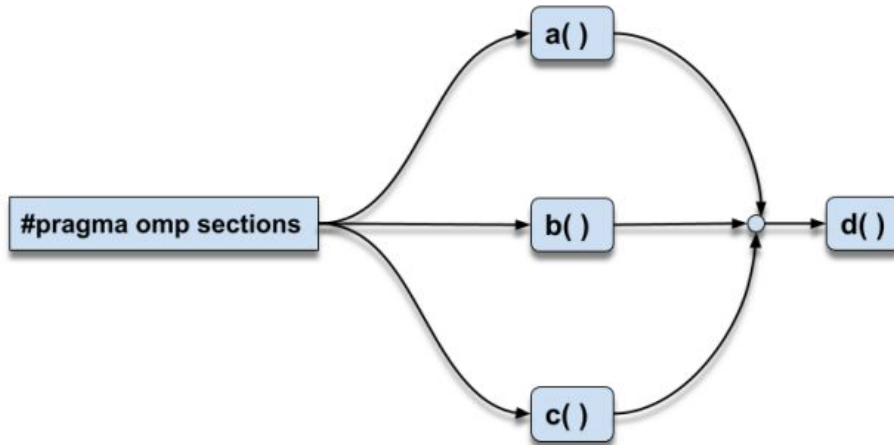
- Decomposição do domínio de dados
 - Cada tarefa trabalha então sobre uma parcela dos dados.
 - A decomposição pode ser cíclica, com pequenas parcelas de dados para cada tarefa, ou em blocos, com o domínio total dividido em grandes blocos para cada tarefa. (*Lembrar da diretiva schedule*)
 - Exemplo: multiplicação de matrizes
- Decomposição Funcional
 - Nesta abordagem, o foco está no tipo de computação realizada, em vez de nos dados manipulados pela computação.
 - O problema é decomposto de acordo com o trabalho a ser realizado.
 - Cada tarefa então realiza uma porção específica do trabalho global.
 - Exemplo: previsão do clima, processamento de sinal.



Particionamento em OpenMP

- Particionamento de dados
 - Diretiva for
- Particionamento de tarefas
 - Diretivas sections e tasks

Diretiva sections



```
#pragma omp sections [ cláusulas ]
{
    # pragma omp section
    {
        a ( ) ; // bloco estruturado
    }
    # pragma omp section
    {
        b ( ) ; // bloco estruturado
    }
    # pragma omp section
    {
        c ( ) ; // bloco estruturado
    }
}
d();
```



Diretiva sections

- A diretiva sections distribui o trabalho entre diversas **threads limitadas a uma região paralela definida**.
- Essa diretiva permite a execução em paralelo de blocos separados de código (por exemplo, diversas subrotinas independentes)
- **Não escalável bem**, porque a quantidade de paralelismo disponível é determinada no código-fonte, sendo portanto pouco utilizada.
- **Cada bloco section é executado apenas por uma thread**
- A diretiva **sections** indica o início de uma construção, que contém diversas diretivas **section**.

```
#pragma omp sections [ cláusulas ]
{
    # pragma omp section
    {
        a ( ) ; // bloco estruturado
    }
    # pragma omp section
    {
        b ( ) ; // bloco estruturado
    }
    # pragma omp section
    {
        c ( ) ; // bloco estruturado
    }
}
d();
```



Diretiva sections

```
• carla@carla-a210:~/workspace/cad$ ./sections
Section 1 - Thread 1
Section 2 - Thread 3
Soma do primeiro loop: 5050
Soma do segundo loop: 338350
• carla@carla-a210:~/workspace/cad$ ./sections
Section 1 - Thread 1
Section 2 - Thread 0
Soma do primeiro loop: 5050
Soma do segundo loop: 338350
• carla@carla-a210:~/workspace/cad$ ./sections
Section 1 - Thread 1
Section 2 - Thread 0
Soma do primeiro loop: 5050
Soma do segundo loop: 338350
```

```
int main(void)
{
    int soma_a = 0;
    int soma_b = 0;

    #pragma omp parallel sections num_threads(4)
    {
        #pragma omp section
        {
            for (int i = 1; i <= 100; ++i)
                soma_a += i;
            printf("Section 1 - Thread %d\n",
                omp_get_thread_num());
        }

        #pragma omp section
        {
            for (int i = 1; i <= 100; ++i)
                soma_b += i * i;
            printf("Section 2 - Thread %d\n",
                omp_get_thread_num());
        }
    }

    printf("Soma do primeiro loop: %d\n", soma_a);
    printf("Soma do segundo loop: %d\n", soma_b);

    return 0;
}
```

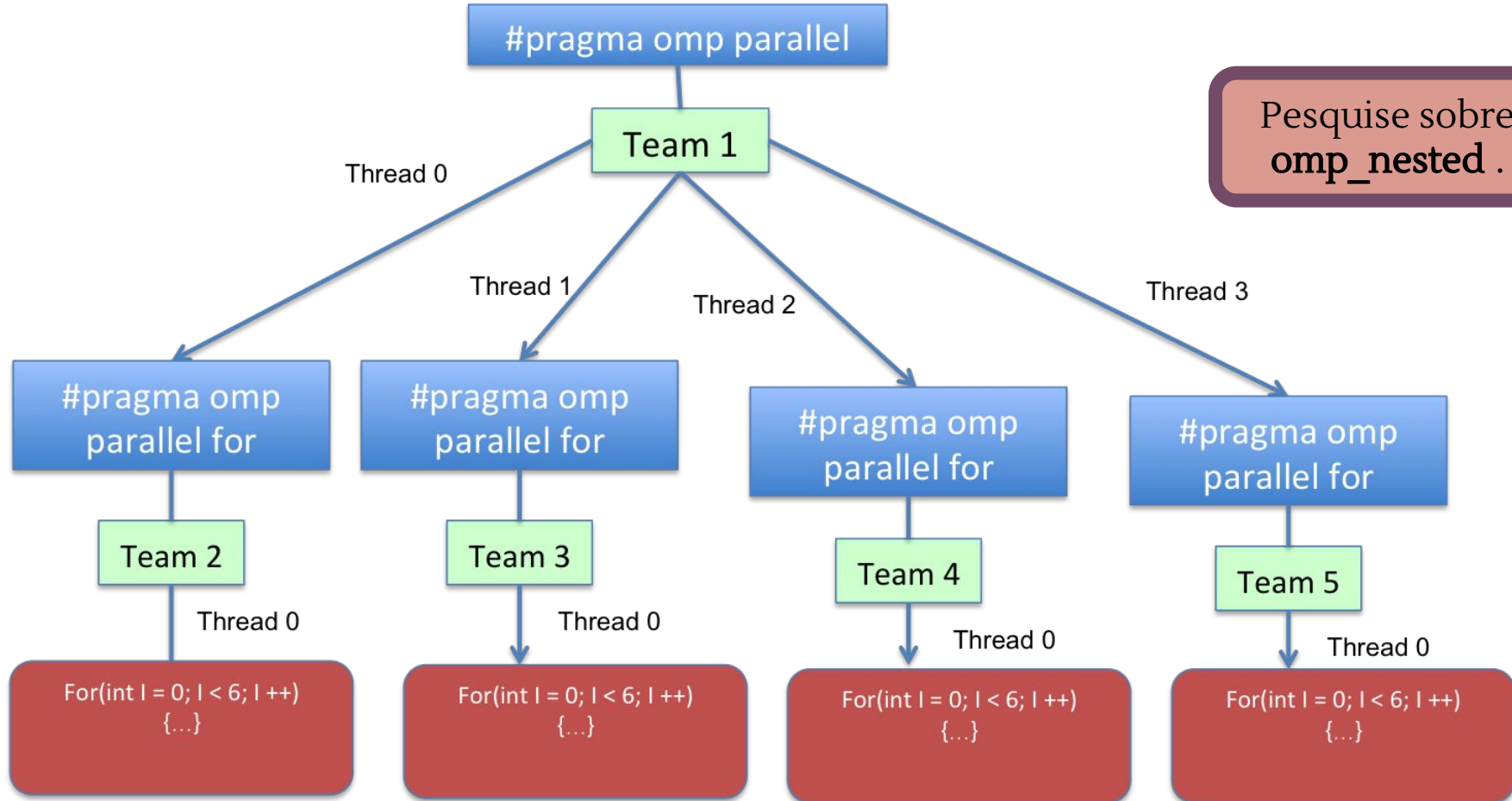
Diretiva sections

```
int main(void)
{
    int soma_a = 0; int soma_b = 0;

    #pragma omp parallel sections num_threads(2)
    {
        #pragma omp section
        {
            #pragma omp parallel num_threads(2)
            {
                for (int i = 1; i <= 100; ++i)
                    soma_a += i;
                printf("Section 1 - Thread %d\n",
                    omp_get_thread_num());
            }
        }

        #pragma omp section
        {
            for (int i = 1; i <= 100; ++i)
                soma_b += i * i;
            printf("Section 2 - Thread %d\n",
                omp_get_thread_num());
        }
    }
    printf("Soma do primeiro loop: %d\n", soma_a);
    printf("Soma do segundo loop: %d\n", soma_b);
    return 0;
}
```







Diretiva sections

```
• carla@carla-a210:~/workspace/cad$ ./sections
Section 2 - Thread 1
Section 1 - Thread 0
Section 1 - Thread 1
Soma do primeiro loop: 10100
Soma do segundo loop: 338350
• carla@carla-a210:~/workspace/cad$ ./sections
Section 2 - Thread 1
Section 1 - Thread 0
Section 1 - Thread 1
Soma do primeiro loop: 10100
Soma do segundo loop: 338350
```

```
int main(void)
{
    int soma_a = 0; int soma_b = 0;
    omp_set_nested(1);

    #pragma omp parallel sections num_threads(2)
    {
        #pragma omp section
        {
            #pragma omp parallel num_threads(2)
            {
                for (int i = 1; i <= 100; ++i)
                    soma_a += i;
                printf("Section 1 - Thread %d\n",
                    omp_get_thread_num());
            }
        }

        #pragma omp section
        {
            for (int i = 1; i <= 100; ++i)
                soma_b += i * i;
            printf("Section 2 - Thread %d\n",
                omp_get_thread_num());
        }
    }

    printf("Soma do primeiro loop: %d\n", soma_a);
    printf("Soma do segundo loop: %d\n", soma_b);
    return 0;
}
```



Diretiva sections

- Algumas regras para a utilização dessas diretivas são:
 - Uma diretiva **section** não pode aparecer fora do bloco estruturado abrangido pela diretiva **sections**;
 - Cada bloco estruturado, guardado por uma diretiva **section**, será executado em paralelo por uma thread distinta;
 - Existe uma barreira implícita definida no final da região paralela associada à diretiva **sections**, se não houver uma cláusula **nowait** especificada

```
#pragma omp sections [ cláusulas ]
{
    # pragma omp section
    {
        a ( ) ; // bloco estruturado
    }
    # pragma omp section
    {
        b ( ) ; // bloco estruturado
    }
    # pragma omp section
    {
        c ( ) ; // bloco estruturado
    }
}
d();
```

Tarefa 8



Implemente um código em OpenMP que realize as seguintes tarefas:

1. Inicialize um vetor que representa uma função qualquer, gerando os valores necessários para o seu processamento.
2. Calcule a integral desse vetor utilizando o método do trapézio.
3. Calcule o vetor que representa a derivada do vetor utilizando o método de diferenças finitas.

As tarefas que não possuem dependência entre si devem ser executadas em paralelo utilizando a diretiva **sections**. Cada tarefa deve utilizar pelo menos duas threads para realizar seu processamento.

Após a conclusão de cada tarefa, o resultado correspondente deve ser impresso por apenas uma thread.

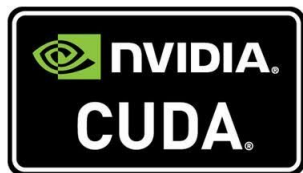
Ao final, discuta as funcionalidades do OpenMP utilizadas na implementação, considerando os seguintes aspectos:

- Como foi explorado o **paralelismo de tarefas**;
- Como foi explorado o **paralelismo de dados**, distribuindo o processamento entre múltiplas threads;
- Quais recursos do OpenMP foram utilizados para permitir o **aninhamento de regiões paralelas** (*nested parallelism*);
- Quais diretivas ou mecanismos do OpenMP foram utilizados para garantir que determinadas operações, como a impressão dos resultados, fossem executadas por **apenas uma thread**;
- Quais mecanismos de **sincronização** estão presentes no código e qual é a função de cada um.

Referências

OpenMP®

MPI



OPEN MPI

Programação Paralela e Distribuída

com MPI, OpenMP e OpenACC
para computação de alto desempenho



Casa do Código | alura

GABRIEL P. SILVA
CALEBE P. BIANCHINI
EVALDO B. COSTA

Programação Paralela para Arquiteturas Híbridas

Com OpenMP OpenACC e MPI

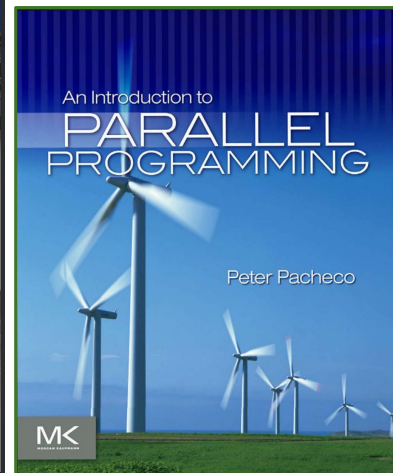


Casa do Código | alura

GABRIEL P. SILVA
CALEBE P. BIANCHINI
EVALDO B. COSTA

An Introduction to PARALLEL PROGRAMMING

Peter Pacheco



MK

